RISC-V Processor with Systolic Array Matrix Multiplier

Complete Project Documentation

Thapar Institute of Engineering & Technology

Digital Design / Computer Architecture

Verilog HDL | Vivado | FPGA

Project Summary

A single-cycle RISC-V processor extended with a custom ISA instruction (matmul) that triggers a dedicated 2x2 systolic array hardware accelerator for matrix multiplication.

1. Project Overview

This project designs and verifies a RISC-V processor extended with a custom hardware accelerator for matrix multiplication. The accelerator uses a systolic array architecture — the same fundamental design used in modern AI chips like Google's TPU.

The processor executes standard RISC-V instructions normally. When a matrix multiplication is needed, a single custom instruction (matmul) triggers the systolic array, which computes the result in hardware while the CPU stalls and waits. Once the computation is complete, the CPU resumes and reads the results from memory.

1.1 What is a Systolic Array?

A systolic array is a network of identical processing elements (PEs) that work in parallel. Data flows through the array in a rhythmic, wave-like pattern — row data flows horizontally, column data flows vertically, and each PE accumulates a partial product as data passes through it.

For an N×N matrix multiplication, a systolic array computes all N^2 output elements simultaneously in just 2N-1 clock cycles, compared to N^3 sequential operations in software. For N=2, that is 3 cycles of active computation instead of 8 sequential multiplications.

1.2 Project Goals

- Design a working single-cycle RISC-V processor supporting addi, add, sub, and, or, slt, sw, lw, and beq
- Extend the ISA with a custom matmul instruction using funct7=0000010
- Build a 2x2 systolic array with a complete controller, memory loader, and result write-back
- Verify every module individually in Vivado XSim
- Verify the complete integrated system end-to-end
- Target FPGA deployment on Xilinx hardware using Vivado

1.3 Tools and Language

Item	Detail
HDL Language	Verilog-2001 (IEEE 1364-2001) — no SystemVerilog constructs
Simulator	Vivado XSim (Xilinx)
Synthesis Target	Xilinx FPGA via Vivado
Simulation Style	Pass/fail testbenches with Tcl console output
ISA Base	RISC-V RV32I subset

Item	Detail
Custom Extension	matmul rd, rs1, rs2 — funct7=0000010, opcode=0110011

2. System Architecture

The complete system is a single-cycle RISC-V CPU with a tightly coupled systolic array accelerator. The CPU and accelerator share the Data_Memory through a priority mux. The accelerator is controlled by a dedicated FSM and fed by a matrix loader module.

2.1 Module Hierarchy

```
Single_Cycle_Top
  ├── PC_Module            — program counter register
  ├── PC_Adder             — PC+4 combinational adder
  ├── Instruction_Memory   — ROM, loads from memfile.hex
  ├── Register_File        — 32x32-bit registers, x0 hardwired zero
  ├── Sign_Extend          — I-type, S-type, B-type immediate extension
  ├── Mux                  — 2:1 mux (register vs immediate for ALU B input)
  ├── ALU                  — ADD, SUB, AND, OR, SLT
  ├── ALU_Decoder          — maps funct3/funct7 to ALUControl
  ├── Control_Unit_Top     — main decoder + done/busy override logic
  ├── Data_Memory          — synchronous write, combinational read
  └── Systolic_Top         — custom accelerator wrapper
        ├── Systolic_Controller — 5-state Moore FSM
        ├── Matrix_Loader       — pre-loads matrices, streams to array
        └── Systolic_Array_2x2  — 2x2 PE grid
              └── PE x4          — multiply-accumulate unit
```

2.2 Data Memory Layout

All matrix data is stored in Data_Memory using a fixed word-aligned layout. The CPU writes matrix values using sw instructions, and reads results using lw instructions after matmul completes.

Byte Address	Word Index	Contents
0	0	A[0][0] — matrix A, row 0, column 0
4	1	A[0][1] — matrix A, row 0, column 1
8	2	A[1][0] — matrix A, row 1, column 0
12	3	A[1][1] — matrix A, row 1, column 1
16	4	B[0][0] — matrix B, row 0, column 0
20	5	B[0][1] — matrix B, row 0, column 1
24	6	B[1][0] — matrix B, row 1, column 0
28	7	B[1][1] — matrix B, row 1, column 1
32	8	C[0][0] — result, written by write-back state machine

Byte Address	Word Index	Contents
36	9	C[0][1] — result
40	10	C[1][0] — result
44	11	C[1][1] — result

2.3 Custom Instruction Encoding

The matmul instruction reuses the standard RISC-V R-type encoding with a unique funct7 value that no standard instruction uses. This allows the existing opcode decoder to detect it without any structural change to the instruction fetch or decode pipeline.

Field	Bits	Value	Meaning
funct7	[31:25]	0000010	Custom matmul identifier
rs2	[24:20]	register number	Source register holding base_B address
rs1	[19:15]	register number	Source register holding base_A address
funct3	[14:12]	000	Standard R-type
rd	[11:7]	register number	Destination — receives C[0][0] at done
opcode	[6:0]	0110011	R-type opcode — same as add/sub/and/or/slt

Comparison with the earlier scalar accelerator (acc instruction, funct7=0000001): the matmul instruction uses funct7=0000010, one bit different. Both encodings can coexist in the same processor. The scalar accelerator was a design stepping stone and is not present in the final system.

3. Module Descriptions

3.1 Processing Element (PE.v)

The PE is the atomic computation unit of the systolic array. Every clock cycle it performs one multiply-accumulate (MAC) operation and forwards its two operands to neighbouring PEs.

Port	Direction	Width	Description
clk	input	1	System clock
rst	input	1	Synchronous reset — clears all outputs
clear	input	1	Zeros accumulator without stopping data flow
A_in	input	32	Matrix A element from the left
B_in	input	32	Matrix B element from above
C_in	input	32	Previous partial sum (fed from own C_out)
A_out	output	32	A_in forwarded rightward to next PE
B_out	output	32	B_in forwarded downward to next PE
C_out	output	32	Updated partial sum: A_in*B_in + C_in

Key design decision — C_in is wired to C_out of the same PE (self-feedback). Because C_out is a registered output, the feedback is: new C_out = A_in * B_in + old C_out. This is not a combinational loop — it is a clocked accumulator. The clear signal zeros C_out for one cycle without interrupting A/B forwarding, so the first data values are not lost.

3.2 Systolic Array (Systolic_Array_2x2.v)

Wires four PE instances into a 2x2 grid. Matrix A flows horizontally left-to-right. Matrix B flows vertically top-to-bottom. Each row of A and each column of B is staggered by one clock cycle (skew) so that the correct pair of elements meets inside each PE at the correct time.

Skew implementation: A_row1 and B_col1 each pass through a one-cycle register (skew delay) before entering the grid. This ensures A[1][0] arrives at PE[1][0] one cycle after A[0][0] arrives at PE[0][0], which is exactly when the array needs it.

Output timing for N=2: C[0][0] is valid after 2 compute cycles, C[0][1] and C[1][0] after 3 cycles, C[1][1] after 3 cycles. All outputs remain stable once valid.

3.3 Systolic Controller (Systolic_Controller.v)

A Moore FSM with five states that orchestrates the entire computation sequence. It does not touch data — it only manages timing through four output signals.

State	Duration	clear	load_en	busy	done
IDLE	Until start	1	0	0	0
LOAD	2*N*N = 8 cycles	1	1	1	0
COMPUTE	N = 2 cycles	0	1	1	0
DRAIN	N = 2 cycles	0	0	1	0
DONE	1 cycle	0	0	0	1

LOAD state lasts 2*N*N = 8 cycles (not 1 cycle) because the Matrix Loader needs this time to pre-fetch all matrix elements from Data_Memory before the array starts. The clear signal stays high during LOAD to prevent partial products from accumulating before valid data arrives. DRAIN lasts N cycles (not N-1) because PE[1][1] — which receives both skew-delayed inputs — needs one extra cycle beyond the others to produce its final result.

3.4 Matrix Loader (Matrix_Loader.v)

The loader bridges Data_Memory and the systolic array. It operates in two internal phases controlled by the load_en signal from the controller.

Phase A — Pre-load (load_en=1, loaded=0)

Reads all 8 matrix elements from Data_Memory one per clock cycle and stores them in 8 internal registers (a00, a01, a10, a11, b00, b01, b10, b11). Data_Memory has one combinational read port — only one element can be read per cycle. After 8 cycles the loaded flag goes high.

Phase B — Stream (load_en=1, loaded=1)

Drives the array inputs combinationally from the internal registers. On column 0: feeds A[0][0], A[1][0], B[0][0], B[0][1]. On column 1: feeds A[0][1], A[1][1], B[1][0], B[1][1]. The output mux is combinational so values are valid in the same cycle that loaded goes high, with no additional latency.

3.5 Systolic Top (Systolic_Top.v)

Wrapper module that connects the controller, loader, and array. Also contains a result write-back state machine that stores all four C values to memory when done fires. The write-back captures C_00..C_11 into registers at the done pulse before the IDLE state reasserts clear and zeros the PE outputs.

3.6 Single Cycle Top (Single_Cycle_Top.v)

The top-level integration module. Key additions beyond the standard single-cycle RISC-V design:

- start_pulse — rising edge detector on is_matmul. Fires a single cycle when the matmul instruction is first fetched.
- Operand capture — base_A_reg and base_B_reg capture RD1_Top and RD2_Top at start_pulse. Critical: the PC advances during stall and RD1/RD2 change to reflect the next instruction. Frozen registers preserve the correct base addresses for the entire computation.
- PC stall — PC_Next = busy ? PC_Top : PCPlus4. While busy=1 the PC is frozen and no new instructions are fetched.
- Branch support — PCSrc = Branch & Zero. PC_Next = busy ? PC_Top : PCSrc ? PCBranch : PCPlus4.
- Data_Memory mux — when busy or write-back active, dm_addr comes from mem_addr_sys not ALUResult.

4. Instruction Set

The processor supports the following instructions, all verified in the final comprehensive test program:

Instruction	Type	Operation	Example	Result
addi rd, rs1, imm	I-type	rd = rs1 + imm	addi x1, x0, 10	x1 = 10
add rd, rs1, rs2	R-type	rd = rs1 + rs2	add x3, x1, x2	x3 = 13
sub rd, rs1, rs2	R-type	rd = rs1 - rs2	sub x4, x1, x2	x4 = 7
and rd, rs1, rs2	R-type	rd = rs1 & rs2	and x5, x1, x2	x5 = 2
or rd, rs1, rs2	R-type	rd = rs1 | rs2	or x6, x1, x2	x6 = 11
slt rd, rs1, rs2	R-type	rd = (rs1<rs2)?1:0	slt x7, x1, x2	x7 = 0
sw rs2, imm(rs1)	S-type	mem[rs1+imm] = rs2	sw x10, 0(x11)	mem[100]=99
lw rd, imm(rs1)	I-type	rd = mem[rs1+imm]	lw x9, 0(x11)	x9 = 99
beq rs1, rs2, off	B-type	if rs1==rs2: PC+=off	beq x1, x1, +8	branch taken
matmul rd,rs1,rs2	Custom	systolic A*B	matmul x22,x0,x9	C stored to mem

5. Assembly Test Program

The final memfile.hex tests every instruction type at least once, culminating in a full matrix multiplication. The CPU does not store matrix data — it is pre-loaded into Data_Memory by the testbench before execution begins.

5.1 Register Assignments

Register	Value	Purpose
x0	0	Hardwired zero — used directly as base_A
x1	10	Test value A for scalar instructions
x2	3	Test value B for scalar instructions
x3	13	add result
x4	7	sub result
x5	2	and result (10 & 3 = 0b1010 & 0b0011 = 0b0010)
x6	11	or result (10 | 3 = 0b1010 | 0b0011 = 0b1011)
x7	0	slt result (10 < 3 = false = 0)
x8	1	slt result (3 < 10 = true = 1)
x9	99	lw result from mem[100]
x10	99	stays 99 — branch taken, overwrite instruction skipped
x15	16	base_B address
x16	32	base_Result address
x17	19	C[0][0] after lw
x18	22	C[0][1] after lw
x19	43	C[1][0] after lw
x20	50	C[1][1] after lw
x21	99	branch landing confirmation
x22	19	rd of matmul — receives C[0][0] via done override

5.2 Program Listing

Address	Hex	Assembly	Comment
0	00a00093	addi x1, x0, 10	x1 = 10
4	00300113	addi x2, x0, 3	x2 = 3
8	002081b3	add x3, x1, x2	x3 = 13

Address	Hex	Assembly	Comment
12	40208233	sub x4, x1, x2	x4 = 7
16	0020f2b3	and x5, x1, x2	x5 = 2
20	0020e333	or x6, x1, x2	x6 = 11
24	0020a3b3	slt x7, x1, x2	x7 = 0
28	00112433	slt x8, x2, x1	x8 = 1
32	06300513	addi x10, x0, 99	x10 = 99
36	06400593	addi x11, x0, 100	x11 = 100
40	00a5a023	sw x10, 0(x11)	mem[100]=99
44	0005a483	lw x9, 0(x11)	x9=99
48	00108463	beq x1, x1, +8	branch taken, skip +4
52	00000513	addi x10, x0, 0	SKIPPED
56	06300a93	addi x21, x0, 99	x21=99 (branch landed)
60	01000793	addi x15, x0, 16	base_B = 16
64	02000813	addi x16, x0, 32	base_Result = 32
68	04f00b33	matmul x22, x0, x15	PC stalls here
72	00000013	nop	wait write-back (1/5)
76	00000013	nop	wait write-back (2/5)
80	00000013	nop	wait write-back (3/5)
84	00000013	nop	wait write-back (4/5)
88	00000013	nop	wait write-back (5/5)
92	00082883	lw x17, 0(x16)	x17 = C[0][0] = 19
96	00482903	lw x18, 4(x16)	x18 = C[0][1] = 22
100	00882983	lw x19, 8(x16)	x19 = C[1][0] = 43
104	00c82a03	lw x20, 12(x16)	x20 = C[1][1] = 50

6. Verification Strategy

Each module was verified independently with a dedicated testbench before being integrated. This bottom-up strategy ensured that bugs were caught at the simplest possible level rather than propagating into the full system.

Phase	Module	Testbench	Tests	Result
Phase 1	PE.v	tb_PE.v	8	PASS 8/8
Phase 2	Systolic_Array_2x2.v	tb_Systolic_2x2.v	20	PASS 20/20
Phase 3	Systolic_Controller.v	tb_Systolic_Controller.v	19	PASS 19/19
Phase 4	Matrix_Loader.v	tb_Matrix_Loader.v	24	PASS 24/24
Phase 5	Systolic_Top.v	tb_Systolic_Top.v	21	PASS 21/21
Phase 6	Single_Cycle_Top.v	Single_Cycle_Top_Tb.v	14	PASS 14/14

Total: 106 individual test checks across 6 testbenches, all passing in Vivado XSim.

7. Challenges and Solutions

The following challenges were encountered during development. Each is documented with the root cause, the symptoms observed, and the exact fix implemented.

7.1 Phase 2 — PE Accumulation Not Working

Challenge

The 2x2 systolic array produced wrong results for all test cases. C[0][0] showed only the last partial product (A[0][1]*B[1][0]) instead of the sum of both partial products.

Root cause: In the first version of Systolic_Array_2x2.v, C_in was wired to 32'd0 (a constant zero) for all four PEs. This meant every clock cycle the PE computed A*B + 0, discarding the previous partial sum entirely. No accumulation occurred.

Solution

Wire C_in of each PE back to its own C_out. Since C_out is a registered output, this creates a clocked feedback path — not a combinational loop. The PE then computes: new C_out = A_in * B_in + previous C_out, which is the correct accumulation.

7.2 Phase 3 — Controller DRAIN Duration

Challenge

C[1][1] was always wrong (showing partial values like 18 instead of 50) while C[0][0], C[0][1], and C[1][0] were all correct. The done pulse fired exactly one cycle too early.

Root cause: The original controller used DRAIN_CYCLES = N-1 = 1. PE[1][1] is the last PE to complete because it receives both A and B through skew delay registers. It needs one additional clock cycle beyond the other PEs to finish its computation. The waveform trace confirmed that C[1][1] reached its correct value exactly one cycle after done fired.

Solution

Change DRAIN_CYCLES from N-1 to N. For N=2 this extends drain from 1 cycle to 2 cycles, giving PE[1][1] the time it needs. This is a fundamental property of any N×N systolic array: the corner PE always requires the most drain time.

7.3 Phase 4 — Matrix Loader Output Timing

Challenge

The Matrix Loader pre-loaded all elements correctly but the first column of outputs appeared one cycle late. The systolic array received zeros on its first compute cycle.

Root cause: The original output mux was registered (inside an always @posedge clk block). When the loaded flag went high at the end of cycle 8, the mux did not drive outputs until the next posedge at cycle 9 — which was already one cycle into COMPUTE.

Solution

Make the output mux combinational (always @(*) instead of always @(posedge clk)). The outputs are then valid in the same cycle that loaded goes high, with zero additional latency. The combinational read of internal registers is safe because all eight element registers are updated only during the LOAD phase.

7.4 Phase 6 — base_A/base_B Captured from Live Register Reads

Challenge

The systolic array computed zeros even though start_pulse fired correctly. Waveform trace showed the loader reading from addresses 32,36,40,44 instead of 0,4,8,12,16,20,24,28.

Root cause: Systolic_Top was receiving base_A and base_B from the live RD1_Top and RD2_Top wires. In the very next cycle after start_pulse fired, the PC advanced and the register file began reading the next instruction's rs1/rs2 fields. RD1_Top changed from 0 to whatever the lw instruction's rs1 was, corrupting the base addresses passed to the loader.

Solution

Add base_A_reg and base_B_reg capture registers in Single_Cycle_Top. On start_pulse, sample RD1_Top and RD2_Top into these registers. Pass the frozen register values to Systolic_Top instead of the live wires. These values remain stable for the entire computation regardless of what the PC does.

7.5 Phase 6 — Data_Memory Reset Wiped Pre-loaded Matrix Data

Challenge

The testbench pre-loaded matrix values into Data_Memory at time #1 but the simulation produced zeros. The loader read correct addresses but received zeros back.

Root cause: Data_Memory has a synchronous reset loop that clears all 1024 words when rst=1. The testbench asserted rst=1 for 50ns. Values written at time #1 (while rst was still high) were wiped by the synchronous clear running every clock cycle during reset.

Solution

Pre-load matrix data after rst=0 releases, not before. In the testbench: wait for #50 (reset period), then wait for one posedge clk, then write the matrix values. By that point the reset loop has stopped executing and the written values persist.

7.6 Phase 6 — C_11 Written as Zero During Write-back

Challenge

C[0][0], C[0][1], C[1][0] were written to memory correctly but C[1][1] was always written as zero.

Root cause: The original write-back state machine drove the live C_11 wire directly. When done fired (cycle 20), the controller transitioned to IDLE on the next cycle, asserting clear=1. This zeroed all PE outputs. The wr_cnt=4 cycle (which writes C_11) fired exactly when clear=1 was active, so it wrote 0 instead of 50.

Solution

Add four result capture registers (c00_reg, c01_reg, c10_reg, c11_reg) in Systolic_Top. At the done posedge, capture all four C values. The write-back state machine reads from these frozen registers, not the live PE wires. The captured values are unaffected by the IDLE state's clear signal.

7.7 Phase 6 — Write-back Collides with lw Instructions

Challenge

After the fixes above, x12, x13, x14 loaded correct values but x15 (C[1][1]) remained zero. The lw instructions were reading wrong addresses.

Root cause: The write-back state machine asserts mem_we_sys=1 for 4 cycles after done. During this time, the Data_Memory address mux gives priority to mem_addr_sys (the write-back address). When the lw instructions executed at cycles overlapping with wr_cnt=4, the CPU's ALUResult address was blocked by the write-back address, so the lw read from the wrong location.

Solution

Insert 5 NOPs between the matmul instruction and the first lw instruction in the assembly program. The write-back takes 4 cycles (wr_cnt 1 through 4). After 5 NOPs, wr_cnt has returned to 0 and mem_we_sys=0, so the Data_Memory address mux is fully returned to CPU control before any lw executes.

7.8 Phase 6 — SLT Missing from ALU_Decoder

Challenge

The slt instruction produced wrong results — instead of 0 or 1 it produced the same result as add.

Root cause: ALU_Decoder had no case for funct3=010 (the SLT encoding). The default case fell through to ALUControl=000 which is ADD. The ALU hardware already had SLT implemented as ALUControl=3'b101, but the decoder never selected it.

Solution

Add one line to ALU_Decoder: 3'b010: ALUControl = 3'b101; This maps the SLT funct3 to the existing SLT operation in the ALU.

7.9 Phase 6 — Branch Not Implemented

Challenge

The beq instruction never branched. The processor always fell through to the next instruction, making the branch test fail.

Root cause: Two separate issues. First, Sign_Extend had no case for ImmSrc=2'b10 (B-type) and returned 32'b0, so PCBranch = PC+0 = PC (infinite loop). Second, the Branch output of Control_Unit_Top was connected to an empty port Branch() in Single_Cycle_Top — it was never wired to the PC mux logic.

Solution

Two fixes applied together. In Sign_Extend: add the B-type case reconstructing the immediate as {In[31], In[7], In[30:25], In[11:8], 1'b0} sign-extended to 32 bits. In Single_Cycle_Top: connect Branch properly — add PCSrc = Branch & Zero, PCBranch = PC_Top + Imm_Ext_Top, and update PC_Next to: busy ? PC_Top : PCSrc ? PCBranch : PCPlus4.

8. Complete File List

All files to be added to the Vivado project:

8.1 Design Sources

File	Type	Status
PE.v	New — Phase 1	Verified
Systolic_Array_2x2.v	New — Phase 2	Verified
Systolic_Controller.v	New — Phase 3	Verified
Matrix_Loader.v	New — Phase 4	Verified
Systolic_Top.v	New — Phase 5/6	Verified
Main_Decoder.v	Updated — Phase 5	Verified
Control_Unit_Top.v	Updated — Phase 5	Verified
ALU_Decoder.v	Updated — Phase 6	SLT added
Sign_Extend.v	Updated — Phase 6	B-type added
Single_Cycle_Top.v	Updated — Phase 5/6	Branch + stall + matmul
Data_Memory.v	Updated — Phase 0	Dead code removed
Register_File.v	Updated — Phase 0	Deterministic reset
PC_Module.v	Original	Unchanged
PC_Adder.v	Original	Unchanged
Instruction_Memory.v	Original	Unchanged
ALU.v	Original	Unchanged
Mux.v	Original	Unchanged

8.2 Simulation Sources

File	Tests	Set as Top to run
tb_PE.v	8	Phase 1 unit test
tb_Systolic_2x2.v	20	Phase 2 unit test
tb_Systolic_Controller.v	19	Phase 3 unit test
tb_Matrix_Loader.v	24	Phase 4 unit test
tb_Systolic_Top.v	21	Phase 5 unit test
Single_Cycle_Top_Tb.v	14	Phase 6 full integration test

8.3 Data Files

File	Contents	Notes
memfile.hex	31 instructions — full test program	Place in Vivado project directory

9. Running Simulation in Vivado

To run any phase test:

1. Open the Vivado project at D:/Verilog/RISC_V_SysArray/
2. In the Sources panel, find the testbench under Simulation Sources > sim_1
3. Right-click the testbench → Set as Top
4. Click Run Simulation → Run Behavioral Simulation
5. In the Tcl console, type: run 2000ns
6. Read PASS/FAIL output in the Tcl console
7. Add signals to the waveform window for visual inspection

For the full integration test (Phase 6), set Single_Cycle_Top_Tb as top and run 2000ns. The Tcl console will print all 14 PASS/FAIL checks grouped by instruction type.

10. Quick Revision Reference

This section summarises the most important facts for fast revision.

Key Numbers

Parameter	Value	Why
N	2	Matrix dimension (2x2 array)
LOAD_CYCLES	8	2*N*N — one read per element, 4A + 4B
COMPUTE_CYCLES	2	N — one cycle per input column
DRAIN_CYCLES	2	N — PE[1][1] needs full N cycles to settle
Total active cycles	12	LOAD+COMPUTE+DRAIN = 8+2+2
NOPs after matmul	5	Write-back takes 4 cycles + 1 margin
matmul funct7	0000010	R-type, distinct from acc (0000001)
base_A	0	Byte address of matrix A in Data_Memory
base_B	16	Byte address of matrix B in Data_Memory
base_Result	32	Byte address where C is written

The Three Control Signals That Drive Everything

- busy: high during LOAD+COMPUTE+DRAIN — freezes PC, suppresses RegWrite/MemWrite
- done: single cycle pulse at end of DONE state — triggers result capture and RegWrite override
- clear: high during IDLE and LOAD — zeros all PE accumulators

What Each Register Holds in the Test Program

x0=0 (base_A), x1=10, x2=3, x3=13(add), x4=7(sub), x5=2(and), x6=11(or), x7=0(slt), x8=1(slt), x9=99(lw), x10=99(branch kept), x15=16(base_B), x16=32(result base), x17=19(C00), x18=22(C01), x19=43(C10), x20=50(C11), x21=99(branch landed), x22=19(matmul rd)

11. Understanding the Partial Products in Waveforms

When viewing simulation waveforms, the C outputs show intermediate values before their final results. These are partial products — not errors. Understanding them confirms the systolic computation is working correctly.

For A = [[1,2],[3,4]] multiplied by B = [[5,6],[7,8]], the expected result is C = [[19,22],[43,50]].

PE	Cycle 1 (partial)	How	Cycle 2 (final)	How
PE[0][0]	5	1*5 + 0 = 5	19	2*7 + 5 = 19
PE[0][1]	6	1*6 + 0 = 6	22	2*8 + 6 = 22
PE[1][0]	15	3*5 + 0 = 15	43	4*7 + 15 = 43
PE[1][1]	18	3*6 + 0 = 18	50	4*8 + 18 = 50

The partial products 5, 6, 15, 18 are visible in the waveform one cycle before the final values. This is not a bug — it is the systolic array working exactly as designed, accumulating one partial product per clock cycle.

Notice that PE[1][1] shows its partial product (18) one cycle later than PE[0][0] shows its partial product (5). This is the skew delay: A_row1 and B_col1 both pass through one-cycle delay registers before entering the grid, so PE[1][1] receives its first data pair one cycle after PE[0][0].

12. FPGA Deployment Notes

The following considerations apply when moving from simulation to FPGA hardware:

12.1 Memory Initialization

In simulation, matrix data is pre-loaded into Data_Memory via testbench hierarchy access (uut.Data_Memory.mem[x] = value). On FPGA this is not possible. The matrix values must be loaded by the CPU using sw instructions before the matmul instruction fires, or loaded via an external interface (UART, switches, etc.).

The current memfile.hex does not include sw instructions for matrix setup because the testbench handles initialization. For FPGA deployment, add sw instructions at the beginning of the program to write matrix values to addresses 0-28 before the matmul instruction.

12.2 Output Display

The debug_out port carries the Result signal — the value being written to the register file each clock cycle. To display results on the FPGA board, connect specific register file outputs to LEDs or a 7-segment display. The four matrix result registers x17-x20 (holding 19, 22, 43, 50) are the natural display candidates.

12.3 Synthesis Observations

The synthesized netlist confirmed correct FPGA mapping: the Register File infers as a RAMB18E1 block RAM primitive, the clock passes through IBUF and BUFG for proper global clock tree distribution, and the combinational logic (ALU, control, muxes) maps into the FPGA LUT fabric.